Classroom Survey Web App - AI Development History

Project and Approach

I developed the Classroom Survey Web App as an individual CSE 4095 Agentic Programming assignment, using Codex as an AI coding agent. My task was to guide implementation from the supplied specifications and verify the resulting full-stack application. I organized the assignment, specification, API, UI, database, and TODO files and used separate, focused conversations rather than treating the project as a one-prompt build. The history below summarizes the actual interactions reflected in the Codex transcript; it is not a verbatim export of every message.

Specification Review and Foundation

My initial requests asked Codex to organize the project files, review the requirements, identify design ambiguities, and produce an implementation plan and decisions record before coding. The first implementation phase introduced the React/TypeScript/Vite foundation, Tailwind, routing, Supabase client configuration, shared layouts, lint and formatting checks, and test tooling. The phase report recorded 20 unit tests and 16 desktop/mobile browser checks passing. After reviewing the staged files, I used Git to make the initial commit e04e8f5 and created the classroom-survey GitHub repository.

Database Design and Security

I next directed Codex to build the specified five-table PostgreSQL schema with foreign keys, indexes, status constraints, RLS policies, and atomic RPC functions. I asked it to ensure students could read only active survey content and could not inspect other students' responses. When instructor permissions were unclear, I asked Codex to inspect the actual migration rather than invent an allowlist. It identified a trusted Supabase Auth app_metadata.role = instructor check. Codex also prepared local schema, authorization, duplicate-submission, and concurrency tests. At that point, it correctly distinguished successful disposable local PostgreSQL tests from remote Supabase verification requiring my account and SQL Editor.

Authentication Debugging

After configuring the project URL and publishable key, I discovered that the instructor login form was disabled and displayed an unfinished-configuration message. I provided Codex with the exact message and requested a root-cause investigation that preserved the existing security model. Its review found that the Phase 1 login page still contained a placeholder which always disabled the form. It implemented Supabase password authentication, session persistence, role checks, protected routes, logout, and recovery-related routes. The reported local checks then passed, while live sign-in with my personal credentials remained a manual test.

Instructor Dashboard and Survey Authoring

I then requested a real instructor dashboard with survey search, filtering, status-appropriate actions, confirmations, and Supabase-backed records. For authoring, I specified multiple-choice and true/false questions, question/choice reordering, validation, draft saving, and preservation of historical responses. The agent implemented an atomic save_survey RPC and a shared editor. During my own attempt to save a two-question Recursion Concept Check survey, the form showed a save failure. I reported that behavior and the example data; Codex traced it to HTTP 404 / PGRST202 for the missing save_surveyRPC in the hosted API and explained how to check the migration and reload the schema cache. I used this diagnosis rather than replacing the editor or weakening RLS.

QR Codes, Anonymous Submission, and Results

I prompted Codex to implement survey activation, one-active-survey enforcement, a presentation page, QR PNG download, and public student submissions. The student flow used the /s/:slug route, required-answer validation, and a database submission RPC. For accidental duplicate protection, Codex added random browser tokens and a uniqueness constraint, while documenting that another device or cleared storage can bypass that limited protection. Later prompts covered result counts and percentages, Recharts charts, historical surveys, duplication, and CSV export. The transcript records several rounds of local tests as these features were added.

Further Debugging, Testing, and Deployment

When survey duplication and historical results failed in the hosted environment, I asked for specific diagnostics. Codex found missing duplicate_survey and get_survey_results RPCs and identified the necessary migrations. I also reported that a QR code would not open on my phone; the local QR used localhost, so the fix introduced a configurable public application origin. The later recorded checks reported 67/67 unit/service tests, 88/88 browser tests across Chromium and Firefox configurations, and successful disposable PostgreSQL migration/security tests. A repository audit checked ignored secrets, source files, specifications, and documentation. The production audit confirmed a Cloudflare-hosted HTTPS app, functioning direct-route refreshes, and tested anonymous security restrictions.

Limitations and Reflection

The production audit did not verify the entire authenticated classroom workflow because the automated environment lacked instructor credentials and an active survey. It specifically left real instructor creation/editing, a physical QR scan, actual student submission persistence, CSV compatibility, and closed/archived results for manual testing. I therefore separate the local test record from live production evidence rather than describing every feature as fully verified. This project taught me to write precise AI requests, inspect reported changes, distinguish frontend bugs from missing database migrations, protect authorization boundaries, and use testing and documentation to evaluate an AI-generated implementation. Codex assisted with significant code

generation, while I remained responsible for directing the process, configuring services, identifying errors, and evaluating the final application.